

INTRODUCTION TO PROGRAMMING

BIC502A | Unit II: Viewing File Contents

Linux Shell, Text Processing and Bash — Explained Practical

A beginner-friendly practical in which every command is explained before it is executed. Follow: What → Why → Command → Breakdown → Run → Observe → Question.

EXAMPLE / BACKGROUND

Unit II focus: text editors, file viewing, searching, text manipulation, piping, writing files using Bash, and Bash scripting.

Colour conventions

BLUE

Command to type and execute.

GREEN

Result/output to inspect.

YELLOW

Important note or common mistake.

GREY

Background, example, or script.

1. Input file

Use the exact teaching dataset name throughout this practical:

EXAMPLE / BACKGROUND

FASTA file: proteins_large.fasta

The supplied teaching dataset contains 12 synthetic protein records for command-line practice.

COMMAND TO EXECUTE

ls proteins_large.fasta

WHAT TO OBSERVE

The filename should be displayed if the FASTA file is in the current working directory.

IMPORTANT NOTE

Do not replace proteins_large.fasta with proteins.fasta. This practical uses the supplied file name consistently.

2. Installation and prerequisites

These are setup steps only; the practical exercises themselves use the Unit II command set.

2.1. Update Ubuntu package information

What does it do? refreshes the local package index.

Why are we using it? software installation should use current package information.

COMMAND TO EXECUTE

```
sudo apt update
```

How to read the command

Part	Meaning
sudo	administrator privileges
apt	Ubuntu package manager
update	refresh package information

WHAT TO OBSERVE

Ubuntu checks the configured software repositories.

Check your understanding: Why is this done before installing software?

2.2. Install required tools

What does it do? installs the editors and text-processing utilities needed for this practical.

Why are we using it? Unit II requires these programs.

COMMAND TO EXECUTE

```
sudo apt install bash nano vim gedit less util-linux grep coreutils gawk sed
```

How to read the command

Part	Meaning
sudo	administrator privileges
apt install	install packages
bash nano vim gedit	shell/editor tools
less util-linux	viewing utilities; more is supplied by util-linux
grep coreutils gawk sed	text-processing tools

WHAT TO OBSERVE

Ubuntu may ask for confirmation.

Check your understanding: Which installed tool is the graphical editor?

3. Viewing the FASTA file

A FASTA record normally begins with a header line beginning with >. Sequence lines follow the header.

EXAMPLE / BACKGROUND

```
>protein_01|KINASE_A|TEACHING_SET
MKTLILAVVTSLACGCGATCGATCGATCGATCGATCGATCG
```

3.1. Display the complete file

What does it do? prints a text file to the terminal.

Why are we using it? we first inspect the complete input.

COMMAND TO EXECUTE

```
cat proteins_large.fasta
```

How to read the command

Part	Meaning
cat	display file contents
proteins_large.fasta	input file

WHAT TO OBSERVE

Headers and sequence lines appear.

Check your understanding: Which character identifies a FASTA header?

3.2. View one screen at a time

What does it do? opens a scrollable file view.

Why are we using it? this is easier for a longer file.

COMMAND TO EXECUTE

```
less proteins_large.fasta
```

How to read the command

Part	Meaning
less	view one screen at a time
proteins_large.fasta	input file

WHAT TO OBSERVE

A scrollable view opens. Press q to leave.

Check your understanding: Why use less instead of cat for a large file?

IMPORTANT NOTE

Press q to quit.

3.3. View page by page

What does it do? displays a file page by page.

Why are we using it? Unit II includes more as another file viewer.

COMMAND TO EXECUTE

```
more proteins_large.fasta
```

How to read the command

Part	Meaning
more	display page by page
proteins_large.fasta	input file

WHAT TO OBSERVE

The file is shown page by page.

Check your understanding: How is this different from cat?

IMPORTANT NOTE

Press q to leave.

3.4. Inspect the beginning

What does it do? shows the beginning of a file.

Why are we using it? we can quickly inspect the first records.

COMMAND TO EXECUTE

```
head proteins_large.fasta
```

How to read the command

Part	Meaning
head	show beginning
proteins_large.fasta	input file

WHAT TO OBSERVE

The first part of the FASTA file appears.

Check your understanding: How many headers are visible?

3.5. Inspect the end

What does it do? shows the end of a file.

Why are we using it? we can inspect the final record quickly.

COMMAND TO EXECUTE

```
tail proteins_large.fasta
```

How to read the command

Part	Meaning
tail	show end
proteins_large.fasta	input file

WHAT TO OBSERVE

The final part appears.

Check your understanding: Can you identify the final record?

3.6. Wrap long lines

What does it do? wraps long lines at a chosen width.

Why are we using it? long sequence lines are easier to read when wrapped.

COMMAND TO EXECUTE

```
fold -w 50 proteins_large.fasta
```

How to read the command

Part	Meaning
fold	wrap lines
-w 50	set width to 50 characters
proteins_large.fasta	input file

WHAT TO OBSERVE

Long lines are wrapped at 50 characters.

Check your understanding: Does fold understand FASTA structure?

IMPORTANT NOTE

fold only changes display layout.

4. Searching with grep and egrep

grep searches text patterns. Start with a simple search, then add options and FASTA-specific patterns.

4.1. Search for KINASE

What does it do? searches for lines containing KINASE.

Why are we using it? we want to find kinase-labelled text.

COMMAND TO EXECUTE

```
grep "KINASE" proteins_large.fasta
```

How to read the command

Part	Meaning
grep	search
"KINASE"	pattern
proteins_large.fasta	input

WHAT TO OBSERVE

Matching lines are displayed.

Check your understanding: Are all matching lines necessarily headers?

4.2. Show line numbers

What does it do? shows the line number of each match.

Why are we using it? line numbers help locate matches.

COMMAND TO EXECUTE

```
grep -n "KINASE" proteins_large.fasta
```

How to read the command

Part	Meaning
grep	search
-n	show line numbers
"KINASE"	pattern
proteins_large.fasta	input

WHAT TO OBSERVE

Matches appear with line numbers.

Check your understanding: What extra information does -n provide?

4.3. Ignore case

What does it do? searches without distinguishing upper/lower case.

Why are we using it? the capitalization may be unknown.

COMMAND TO EXECUTE

```
grep -i "kinase" proteins_large.fasta
```

How to read the command

Part	Meaning
grep	search
-i	ignore case
"kinase"	pattern
proteins_large.fasta	input

WHAT TO OBSERVE

Matches are found regardless of capitalization.

Check your understanding: When is -i useful?

4.4. Count matching lines

What does it do? counts matching lines.

Why are we using it? we need a numerical result.

COMMAND TO EXECUTE

```
grep -c "KINASE" proteins_large.fasta
```

How to read the command

Part	Meaning
grep	search
-c	count matching lines
"KINASE"	pattern
proteins_large.fasta	input

WHAT TO OBSERVE

One number is printed.

Check your understanding: Does -c count records or matching lines?

4.5. Find FASTA headers

What does it do? selects lines beginning with >.

Why are we using it? FASTA headers begin with >.

COMMAND TO EXECUTE

```
grep "^>" proteins_large.fasta
```

How to read the command

Part	Meaning
grep	search
"^>"	^ = start of line; > is first character
proteins_large.fasta	input

WHAT TO OBSERVE

Only header lines are displayed.

Check your understanding: Why does ^> select headers?

4.6. Count FASTA records

What does it do? counts FASTA header lines.

Why are we using it? one header represents one FASTA record.

COMMAND TO EXECUTE

```
grep -c "^>" proteins_large.fasta
```

How to read the command

Part	Meaning
grep	search
-c	count matches
"^>"	header pattern
proteins_large.fasta	input

WHAT TO OBSERVE

The result should be 12.

Check your understanding: Why can header count equal record count?

4.7. Search for HGG

What does it do? searches for the text HGG.

Why are we using it? the teaching dataset includes search motifs.

COMMAND TO EXECUTE

```
grep "HGG" proteins_large.fasta
```

How to read the command

Part	Meaning
grep	search
"HGG"	pattern
proteins_large.fasta	input

WHAT TO OBSERVE

Lines containing HGG are displayed.

Check your understanding: Is grep interpreting HGG biologically or as text?

4.8. Search for KINASE or ENZYME

What does it do? uses extended pattern matching for alternatives.

Why are we using it? one search can find either label.

COMMAND TO EXECUTE

```
egrep "KINASE|ENZYME" proteins_large.fasta
```

How to read the command

Part	Meaning
egrep	extended regular-expression search
"KINASE ENZYME"	means either pattern
proteins_large.fasta	input

WHAT TO OBSERVE

Lines containing either label are displayed.

Check your understanding: What does | mean inside the pattern?

5. Data manipulation

5.1. Extract a field with cut

What does it do? extracts selected fields using a delimiter.

Why are we using it? the FASTA headers use | as a separator.

COMMAND TO EXECUTE

```
grep "^>" proteins_large.fasta | cut -d "|" -f2
```

How to read the command

Part	Meaning
grep "^>"	select headers
	pipe output to cut
cut	extract fields
-d " "	use as delimiter
-f2	select field 2

WHAT TO OBSERVE

Protein names are displayed.

Check your understanding: Which option defines the delimiter?

IMPORTANT NOTE

Notice that the pipe between commands is different from the | used as a delimiter.

5.2. Join lines with paste

What does it do? joins corresponding lines from files side by side.

Why are we using it? this demonstrates the Unit II paste operation.

COMMAND TO EXECUTE

```
paste file1.txt file2.txt
```

How to read the command

Part	Meaning
paste	join lines
file1.txt	first input
file2.txt	second input

WHAT TO OBSERVE

Corresponding lines appear beside one another.

Check your understanding: What does paste do to the two files?

IMPORTANT NOTE

Use two small instructor-provided text files if these files are not already present.

5.3. Transform displayed text with sed

What does it do? searches and transforms text in the output.

Why are we using it? we can demonstrate replacement without changing the original file.

COMMAND TO EXECUTE

```
sed 's/KINASE/PROTEIN_KINASE/g' proteins_large.fasta
```

How to read the command

Part	Meaning
sed	stream editor
s	substitute
KINASE	find this
PROTEIN_KINASE	replace with this
g	replace all occurrences on a matching line
proteins_large.fasta	input

WHAT TO OBSERVE

KINASE is displayed as PROTEIN_KINASE.

Check your understanding: Has the original file been changed?

IMPORTANT NOTE

The command displays transformed text; it does not save the change to the original.

5.4. Print FASTA headers with awk

What does it do? processes text line by line and prints selected lines.

Why are we using it? awk can select lines using a pattern.

COMMAND TO EXECUTE

```
awk '/^>/{print $0}' proteins_large.fasta
```

How to read the command

Part	Meaning
awk	process lines
/^>/{	select headers
print \$0}	print complete line
proteins_large.fasta	input

WHAT TO OBSERVE

FASTA headers are displayed.

Check your understanding: What does \$0 represent?

5.5. Count headers with awk

What does it do? performs a simple count while reading the file.

Why are we using it? this shows that awk can calculate as it processes text.

COMMAND TO EXECUTE

```
awk '/^>/{count++} END{print "Number of proteins:", count}' proteins_large.fasta
```

How to read the command

Part	Meaning
awk	process file
/^> /	identify headers
count++	increase counter
END{...}	run after reading
proteins_large.fasta	input

WHAT TO OBSERVE

The number of protein records is printed.

Check your understanding: What causes count to increase?

6. Piping and writing files

The pipe | passes output from one command to another. Redirection sends output into a file.

6.1. Count headers with a pipe

What does it do? connects header selection to line counting.

Why are we using it? we first select headers and then count them.

COMMAND TO EXECUTE

```
grep "^>" proteins_large.fasta | wc -l
```

How to read the command

Part	Meaning
grep "^>"	select headers
	pass output to next command
wc -l	count lines

WHAT TO OBSERVE

A single number is printed: 12.

Check your understanding: What is the role of |?

IMPORTANT NOTE

The pipe is an operator, not a command.

6.2. Count KINASE headers

What does it do? combines header matching and line counting.

Why are we using it? we want KINASE-labelled records.

COMMAND TO EXECUTE

```
grep "^>.*KINASE" proteins_large.fasta | wc -l
```

How to read the command

Part	Meaning
grep	search
"^>.*KINASE"	header beginning with > and containing KINASE
	pass matches onward
wc -l	count lines

WHAT TO OBSERVE

A single count is printed.

Check your understanding: Why anchor the pattern with ^>?

6.3. Write headers to a file

What does it do? redirects output into a file.

Why are we using it? we want a reusable header list.

COMMAND TO EXECUTE

```
grep "^>" proteins_large.fasta > protein_headers.txt
```

How to read the command

Part	Meaning
grep "^>"	select headers
>	redirect output
protein_headers.txt	destination file

WHAT TO OBSERVE

protein_headers.txt is created with the headers.

Check your understanding: What is stored in the file?

IMPORTANT NOTE

> writes output to the file and replaces existing contents.

6.4. Append output to a file

What does it do? adds output to the end of an existing file.

Why are we using it? we want to demonstrate append rather than replace.

COMMAND TO EXECUTE

```
grep "KINASE" proteins_large.fasta >> protein_headers.txt
```

How to read the command

Part	Meaning
grep "KINASE"	select matching lines
>>	append output
protein_headers.txt	destination

WHAT TO OBSERVE

KINASE lines are added after existing contents.

Check your understanding: What is the difference between > and >>?

7. Text editors

7.1. Open a file with nano

What does it do? opens a simple terminal editor.

Why are we using it? nano is suitable for beginners.

COMMAND TO EXECUTE

```
nano notes.txt
```

How to read the command

Part	Meaning
nano	terminal editor
notes.txt	file to open/create

WHAT TO OBSERVE

The file opens in the terminal.

Check your understanding: Which keys save and exit?

IMPORTANT NOTE

Ctrl+O saves; Ctrl+X exits.

7.2. Open a file with vim

What does it do? opens a terminal editor with different modes.

Why are we using it? vim is part of Unit II.

COMMAND TO EXECUTE

```
vim notes.txt
```

How to read the command

Part	Meaning
vim	terminal editor
notes.txt	file

WHAT TO OBSERVE

The file opens in vim.

Check your understanding: What is the beginner edit/save sequence?

IMPORTANT NOTE

Use i, Esc, then :wq.

7.3. Open a file with gedit

What does it do? opens a graphical editor.

Why are we using it? gedit provides a graphical editing option.

COMMAND TO EXECUTE

```
gedit notes.txt
```

How to read the command

Part	Meaning
gedit	graphical editor
notes.txt	file

WHAT TO OBSERVE

The file opens graphically if available.

Check your understanding: Is gedit guaranteed on every system?

IMPORTANT NOTE

gedit may not be installed on every system.

8. Bash scripting

A Bash script is a text file containing shell commands that can be run together. The script below counts FASTA headers.

EXAMPLE / BACKGROUND

```
#!/bin/bash
file="$1"
echo "File: $file"
echo "Number of proteins:"
grep -c "^>" "$file"
```

8.1. Create the Bash script

What does it do? opens a file for writing the script.

Why are we using it? we need a reusable FASTA-summary workflow.

COMMAND TO EXECUTE

```
nano fasta_summary.sh
```

How to read the command

Part	Meaning
nano	open terminal editor
fasta_summary.sh	script file

WHAT TO OBSERVE

Enter the script above, save it, and exit.

Check your understanding: What is the purpose of `#!/bin/bash`?

8.2. Run the Bash script

What does it do? runs the script and supplies the FASTA filename as an argument.

Why are we using it? the script needs an input file.

COMMAND TO EXECUTE

```
bash fasta_summary.sh proteins_large.fasta
```

How to read the command

Part	Meaning
bash	run using Bash
fasta_summary.sh	script
proteins_large.fasta	argument supplied to the script

WHAT TO OBSERVE

The script prints the filename and protein count.

Check your understanding: Which part becomes `$1`?

9. Independent practical

Perform these tasks independently using `proteins_large.fasta`. Keep your script and output files for submission.

1. Display the complete FASTA file with `cat`.
2. Compare `less` and `more`.
3. Inspect the beginning and end with `head` and `tail`.
4. Wrap the display using `fold -w 50`.
5. Search for KINASE, kinase and HGG using `grep`.
6. Count FASTA records with `grep -c '^>'`.
7. Use `grep -n` and explain the additional information.
8. Use `egrep` to search for KINASE or ENZYME.
9. Extract protein names using `cut`.
10. Use `paste` with two small text files supplied by the instructor.
11. Use `sed` to replace KINASE with PROTEIN_KINASE in displayed output.
12. Use `awk` to print and count FASTA headers.
13. Use a pipe to connect `grep` output to `wc -l`.
14. Save headers with `>` and append with `>>`.
15. Open `notes.txt` with `nano`, `vim` and `gedit`.
16. Write and run `fasta_summary.sh`.

IMPORTANT NOTE

Assessment is based on understanding. Be able to explain each command, option, pattern, delimiter, pipe and redirection operator.

10. Quick reference

Command / operator	Purpose
<code>cat</code>	display a file
<code>less</code>	view one screen at a time
<code>more</code>	view page by page
<code>head</code>	show beginning
<code>tail</code>	show end
<code>fold</code>	wrap long lines
<code>grep</code>	search text
<code>egrep</code>	extended pattern search
<code>cut</code>	extract fields
<code>paste</code>	join lines
<code>sed</code>	search and transform text
<code>awk</code>	process lines and calculate
<code> </code>	send output to another command
<code>></code>	write/redirect output
<code>>></code>	append output

Command / operator	Purpose
nano	terminal editor
vim	terminal editor
gedit	graphical editor
bash	run scripts